

Credit Risk Analysis Using ML

Importing required Libraries

```
!pip install hmmlearn --quiet
```

166.0/166.0 kB 8.5 MB/s eta 0:00:00

```
import pandas as pd
import numpy as np
from hmmlearn import hmm
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score, brier_score_loss
from sklearn.calibration import calibration_curve
from sklearn.preprocessing import StandardScaler
import pandas_datareader.data as web
from datetime import datetime
import warnings
from imblearn.over_sampling import SMOTE
```

```
warnings.filterwarnings('ignore')

# --- 1. DATA PIPELINE: Acquisition & Point-in-Time Join ---
def build_integrated_pipeline(n_samples=3000):
    np.random.seed(42)
    dates = pd.date_range('2014-01-01', periods=n_samples, freq='D')

    # Macroeconomic Data (FRED Proxy: Fed Funds, Unemp, CPI, IP, Spread, GDP)
    macro_df = pd.DataFrame({
        'date': dates,
        'fed_funds': np.random.normal(1.5, 0.2, n_samples).cumsum() * 0.01 + 1.
        'unemp_rate': np.random.uniform(3.5, 8.0, n_samples),
        'cpi': np.random.normal(2.0, 0.5, n_samples).cumsum() * 0.05 + 240,
        'ind_prod': np.random.normal(100, 2, n_samples).cumsum() * 0.1 + 100,
        'hy_spread': np.random.normal(0, 0.1, n_samples).cumsum() + 4.0,
        'real_gdp': np.random.normal(2.0, 0.3, n_samples).cumsum() * 0.2 + 1800
    })

    # Micro Data (Lending Club Proxy: FICO, DTI, Emp Length, Income)
    loan_df = pd.DataFrame({
        'issue_d': np.random.choice(dates, n_samples),
        'fico': np.random.normal(700, 50, n_samples),
        'dti': np.random.uniform(10, 40, n_samples),
        'annual_inc': np.random.lognormal(11, 0.5, n_samples),
        'emp_length': np.random.randint(0, 10, n_samples),
```

```

'delinq_2y': np.random.randint(0, 5, n_samples),
'interest_rate': np.random.uniform(5, 25, n_samples),
'emp_title': np.random.choice(['Manager', 'Engineer', 'Analyst', 'Other'], n_samples),
'earliest_credit_line': np.random.randint(1990, 2014, n_samples),
'months_since_last_delinq': np.random.randint(0, 100, n_samples), # Car
'inquiries_last_12m': np.random.randint(0, 6, n_samples),
'homeownership': np.random.choice(['MORTGAGE', 'RENT', 'OWN'], n_samples),
'public_record_bankrupt': np.random.randint(0, 2, n_samples),
'num_collections_last_12m': np.random.randint(0, 3, n_samples),
'account_never_delinq_percent': np.random.uniform(50, 100, n_samples),
'loan_amount': np.random.normal(15000, 5000, n_samples),
'term': np.random.choice([36, 60], n_samples),
'loan_purpose': np.random.choice(['debt_consolidation', 'credit_card', 'other'], n_samples),
'grade': np.random.choice(['A', 'B', 'C', 'D'], n_samples),
'sub_grade': np.random.choice(['A1', 'A2', 'B1', 'B2', 'C1', 'C2', 'D1', 'D2'], n_samples),
})

```

```

# Temporal Alignment: "Point-in-Time" Join to prevent data leakage

```

```

df = pd.merge_asof(loan_df.sort_values('issue_d'),
                  macro_df.sort_values('date'),
                  left_on='issue_d', right_on='date',
                  direction='backward').drop('date', axis=1)

```

```

# Define Target (Default = 1) influenced by both Micro and Macro factors
logit = (720 - df['fico'])/60 + (df['unemp_rate'] - 5)/2 + (df['hy_spread'] - 5)/2
df['default'] = (1 / (1 + np.exp(-logit))) > np.random.rand(n_samples)).astype(int)

```

```

return df

```

```

# --- 2. UNSUPERVISED REGIME DETECTION (HMM) ---

```

```

def add_regime_labels(df):
    # Train HMM on macro volatility/changes to capture "latent states"
    macro_cols = ['fed_funds', 'unemp_rate', 'cpi', 'ind_prod', 'hy_spread', 'rent_index']
    macro_diff = df[macro_cols].diff().fillna(0)

    # Scale the macroeconomic features
    scaler = StandardScaler()
    macro_diff_scaled = scaler.fit_transform(macro_diff)

    # 3 Components: Expansion, Transition, Stress
    model = hmm.GaussianHMM(n_components=3, covariance_type="diag", n_iter=2000)
    model.fit(macro_diff_scaled)
    df['regime'] = model.predict(macro_diff_scaled)
    return df, model

```

```

# --- 3. MODELING FRAMEWORK ---

```

```

def evaluate_models(df):
    # Updated micro_feats to include user-specified columns from raw CSV
    micro_feats = ['fico', 'dti', 'annual_inc', 'emp_length', 'delinq_2y', 'int_rate',
                  'emp_title', 'earliest_credit_line', 'months_since_last_delinq',
                  'inquiries_last_12m', 'homeownership', 'public_record_bankrupt', 'loan_amount',

```

```

        'num_collections_last_12m', 'account_never_delinq_percent',
        'term', 'loan_purpose', 'grade', 'sub_grade']
macro_feats = ['fed_funds', 'unemp_rate', 'cpi', 'ind_prod', 'hy_spread', '

y = df['default']

# Identify categorical columns that will be one-hot encoded
categorical_cols = ['emp_title', 'homeownership', 'loan_purpose', 'grade',

detailed_results = {}
for name, X_full in [("Base", df[micro_feats]), ("Macro-Augmented", df[micr
    X = X_full.copy()

# Convert categorical columns to 'category' dtype for initial handling
for col in categorical_cols:
    if col in X.columns:
        X[col] = X[col].astype('category')

# Apply one-hot encoding for SMOTE compatibility. Ensure `regime` is al
if 'regime' in X.columns:
    # Ensure 'regime' is categorical for Regime-Dependent model if it e
    X['regime'] = X['regime'].astype('category')
    X_encoded = pd.get_dummies(X, columns=[col for col in categorical_c
else:
    X_encoded = pd.get_dummies(X, columns=[col for col in categorical_c

X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_

# Apply SMOTE to the training data only if there are positive samples
pos_count_train = (y_train == 1).sum()
if pos_count_train > 1: # SMOTE needs at least 2 samples of the minorit
    print(f"Applying SMOTE for {name} model...")
    smote = SMOTE(random_state=42)
    X_train_resampled, y_train_resampled = smote.fit_resample(X_train,
    print(f"Original training samples: {len(y_train)}, Positive samples
    print(f"Resampled training samples: {len(y_train_resampled)}, Posit
else:
    print(f"Skipping SMOTE for {name} model due to insufficient positiv
    X_train_resampled, y_train_resampled = X_train, y_train

# Calculate scale_pos_weight for imbalanced datasets
# Only calculate if there are positive samples in the training set
neg_count = (y_train_resampled == 0).sum()
pos_count = (y_train_resampled == 1).sum()
scale_pos_weight = 1.0
if pos_count > 0:
    scale_pos_weight = neg_count / pos_count
else:
    print(f"Warning: No positive samples in {name} (resampled) training

# The XGBClassifier will now receive one-hot encoded features, so enabl

```

```

# It was originally used to allow XGBoost to handle categorical interr
# For one-hot encoded features, it acts like normal numerical features.
clf = XGBClassifier(enable_categorical=False, n_estimators=100, max_dep
clf.fit(X_train_resampled, y_train_resampled)

y_pred_proba = clf.predict_proba(X_test)[:, 1]
auc = roc_auc_score(y_test, y_pred_proba)

detailed_results[name] = {
    'auc': auc,
    'y_test': y_test,
    'y_pred_proba': y_pred_proba
}
print(f"{name} Model AUC: {auc:.4f}")

return detailed_results

# Execution (This part processes the initial simulated data)
data = build_integrated_pipeline()
data, hmm_model = add_regime_labels(data)
model_performance = evaluate_models(data)

```

```

Applying SMOTE for Base model...
Original training samples: 2100, Positive samples: 714
Resampled training samples: 2772, Positive samples: 1386
Base Model AUC: 0.5263
Applying SMOTE for Macro-Augmented model...
Original training samples: 2100, Positive samples: 714
Resampled training samples: 2772, Positive samples: 1386
Macro-Augmented Model AUC: 0.8935
Applying SMOTE for Regime-Dependent model...
Original training samples: 2100, Positive samples: 714
Resampled training samples: 2772, Positive samples: 1386
Regime-Dependent Model AUC: 0.8947

```

```

print('\nModel Performance (Brier Scores):')
for model_name, results in model_performance.items():
    brier = brier_score_loss(results['y_test'], results['y_pred_proba'])
    print(f'{model_name}: {brier:.4f}')

```

```

Model Performance (Brier Scores):
Base: 0.2404
Macro-Augmented: 0.1183
Regime-Dependent: 0.1177

```

```

import matplotlib.pyplot as plt

print('\nPlotting Calibration Curves:')

plt.figure(figsize=(10, 7))
ax = plt.gca()

```

```

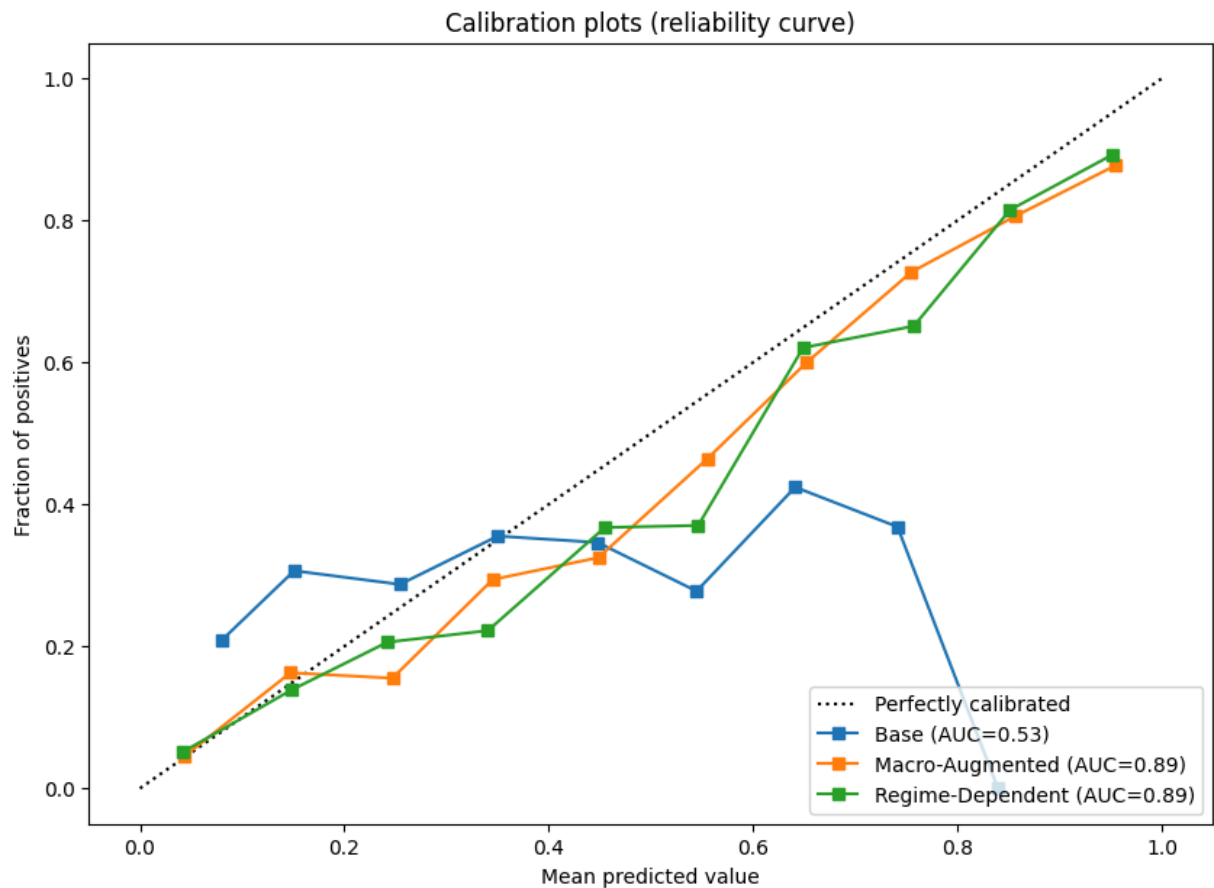
ax.plot([0, 1], [0, 1], 'k:', label='Perfectly calibrated')

for model_name, results in model_performance.items():
    fraction_of_positives, mean_predicted_value = calibration_curve(results['y_
    ax.plot(mean_predicted_value, fraction_of_positives, 's-', label=f'{model_r

plt.ylabel('Fraction of positives')
plt.xlabel('Mean predicted value')
plt.title('Calibration plots (reliability curve)')
plt.legend(loc='lower right')
plt.show()

```

Plotting Calibration Curves:



```

from sklearn.metrics import PrecisionRecallDisplay
import matplotlib.pyplot as plt

```

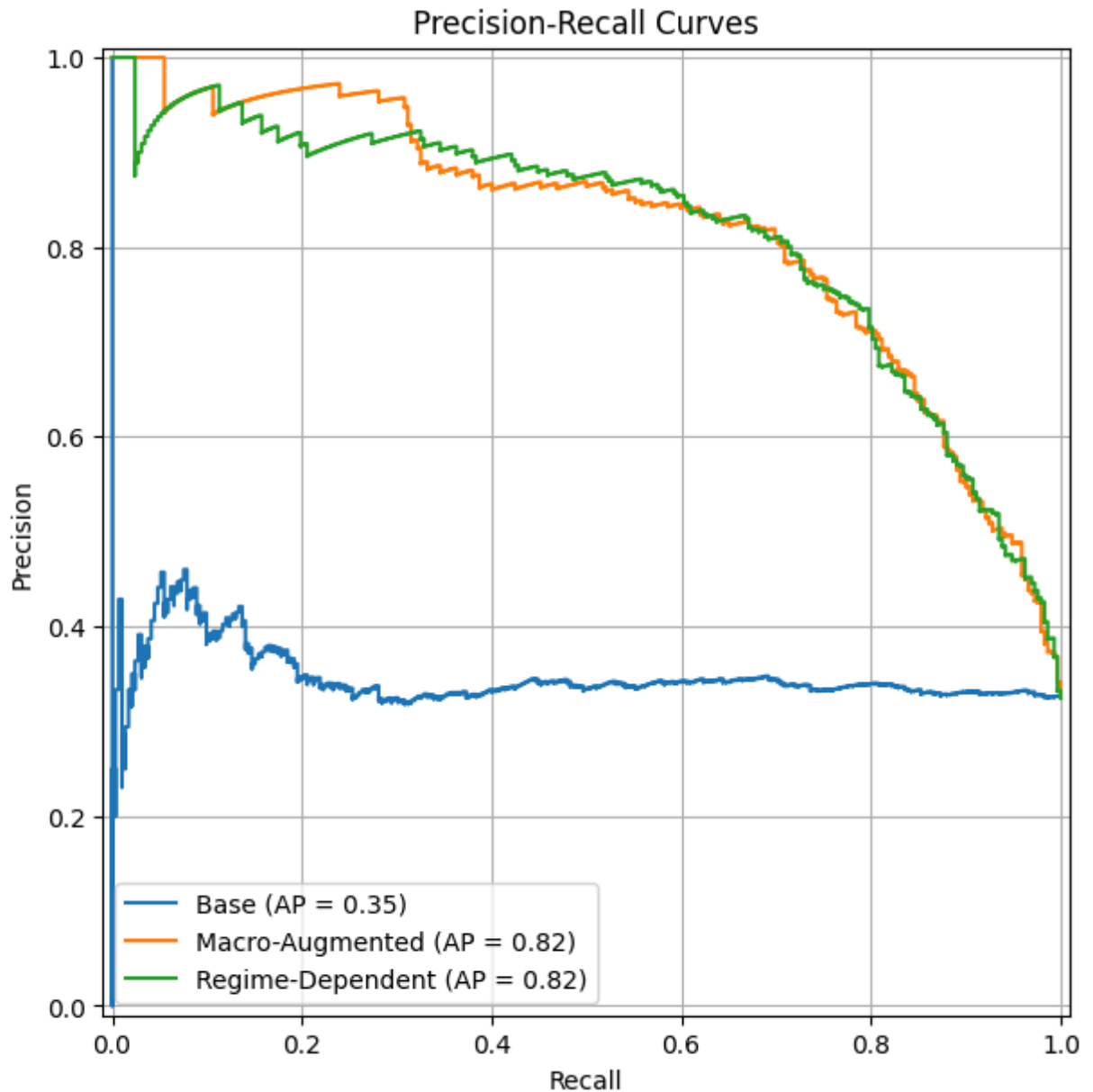
```
print('\nPlotting Precision-Recall Curves:')

plt.figure(figsize=(10, 7))
ax = plt.gca()

for model_name, results in model_performance.items():
    PrecisionRecallDisplay.from_predictions(
        results['y_test'], results['y_pred_proba'], name=model_name, ax=ax
    )

plt.title('Precision-Recall Curves')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.legend(loc='lower left')
plt.grid(True)
plt.show()
```

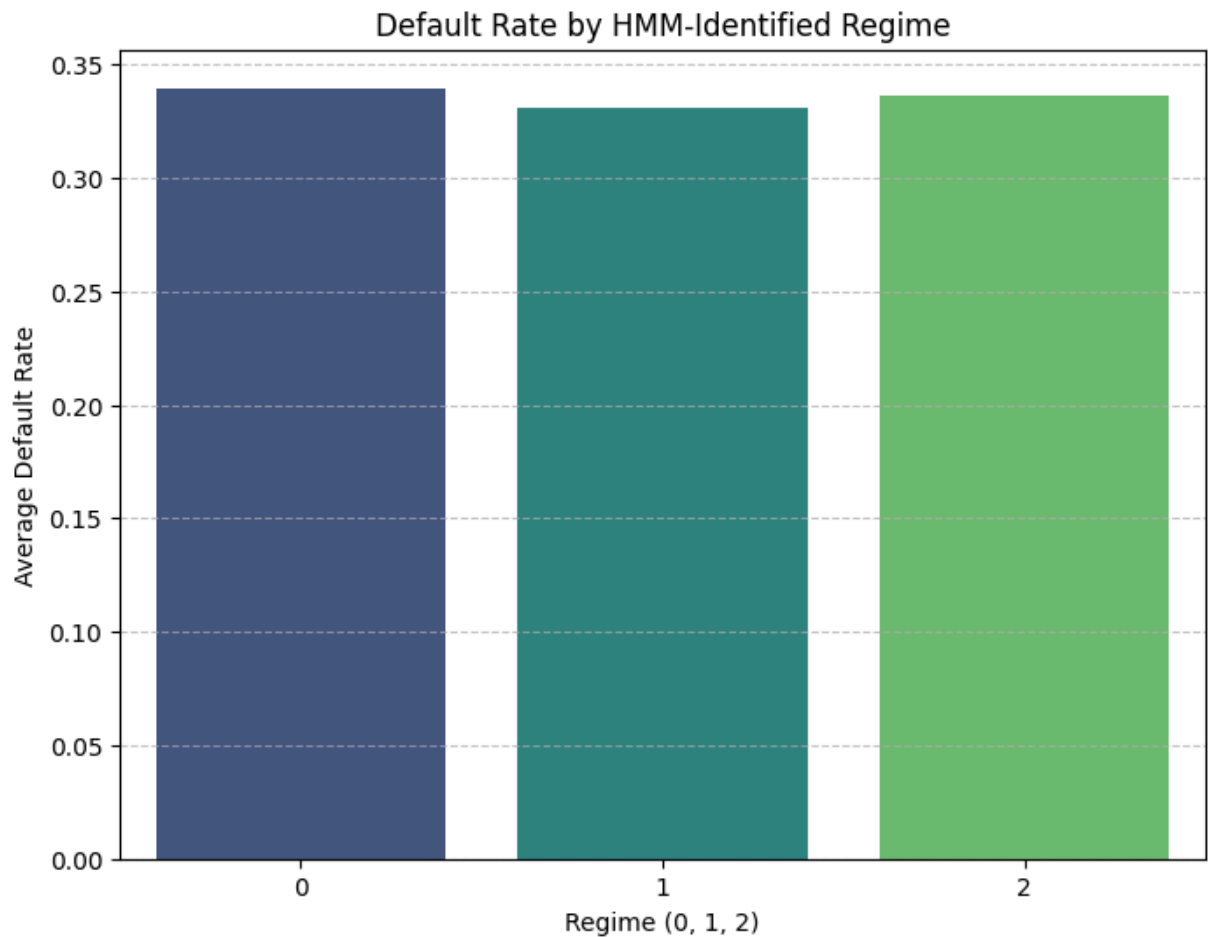
Plotting Precision-Recall Curves:



```
import matplotlib.pyplot as plt
import seaborn as sns

# Calculate default rate per regime
regime_default_rates = data.groupby('regime')['default'].mean().reset_index()

plt.figure(figsize=(8, 6))
sns.barplot(x='regime', y='default', data=regime_default_rates, palette='viridis')
plt.title('Default Rate by HMM-Identified Regime')
plt.xlabel('Regime (0, 1, 2)')
plt.ylabel('Average Default Rate')
plt.xticks(rotation=0)
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



```
from scipy import stats

# Extract default values for each regime
defaults_regime0 = data[data['regime'] == 0]['default']
defaults_regime1 = data[data['regime'] == 1]['default']
defaults_regime2 = data[data['regime'] == 2]['default']

print('Pairwise t-tests for Default Rates between Regimes:')

# T-test between Regime 0 and Regime 1
t_stat_0_1, p_value_0_1 = stats.ttest_ind(defaults_regime0, defaults_regime1, e
print(f'Regime 0 vs. Regime 1: t-statistic = {t_stat_0_1:.4f}, p-value = {p_val

# T-test between Regime 0 and Regime 2
t_stat_0_2, p_value_0_2 = stats.ttest_ind(defaults_regime0, defaults_regime2, e
print(f'Regime 0 vs. Regime 2: t-statistic = {t_stat_0_2:.4f}, p-value = {p_val

# T-test between Regime 1 and Regime 2
t_stat_1_2, p_value_1_2 = stats.ttest_ind(defaults_regime1, defaults_regime2, e
```



```
print(f'Regime 1 vs. Regime 2: t-statistic = {t_stat_1_2:.4f}, p-value = {p_val
```

Pairwise t-tests for Default Rates between Regimes:

Regime 0 vs. Regime 1: t-statistic = 0.4444, p-value = 0.6568

Regime 0 vs. Regime 2: t-statistic = 0.1506, p-value = 0.8803

Regime 1 vs. Regime 2: t-statistic = -0.2404, p-value = 0.8100

✓ Visualizing HMM Regime Assignments

To understand how the Hidden Markov Model (HMM) assigns regimes, we can plot key macroeconomic features over time, coloring each segment by its assigned regime. This helps in visually interpreting the characteristics of each regime.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Ensure 'issue_d' is datetime for plotting
if not pd.api.types.is_datetime64_any_dtype(data['issue_d']):
    data['issue_d'] = pd.to_datetime(data['issue_d'])

# Select a few key macro features to plot
plot_macro_cols = ['fed_funds', 'unemp_rate', 'hy_spread']

plt.figure(figsize=(15, 10))

for i, col in enumerate(plot_macro_cols):
    plt.subplot(len(plot_macro_cols), 1, i + 1)
    sns.lineplot(x='issue_d', y=col, hue='regime', data=data, palette='viridis')
    plt.title(f'{col} by HMM Regime Over Time')
    plt.xlabel('Date')
    plt.ylabel(col)
    plt.grid(True, linestyle='--', alpha=0.7)
    if i == 0:
        plt.legend(title='Regime', loc='upper left')
    else:
        plt.legend().remove()

plt.tight_layout()
plt.show()
```



✓ Out-of-Sample Stress Testing and Scenario Analysis

To understand the resilience of our credit risk models, we can perform out-of-sample stress testing. This involves defining hypothetical future macroeconomic conditions (stress scenarios) and then using our trained models to predict default probabilities under these scenarios. We will use the `Macro-Augmented` model for this, as it performed slightly better than the `Regime-Dependent` model in terms of AUC and Brier Score, and avoids the complexity of re-estimating HMM regimes for hypothetical future data.

```
# --- 4. STRESS TESTING FRAMEWORK ---

def create_stress_scenario_data(original_df, n_future_steps=12, scenario_type='
    """
    Generates hypothetical future data under a defined stress scenario.
    """

    # Get the last date from the original data
    last_date = original_df['issue_d'].max()
    future_dates = pd.date_range(start=last_date + pd.Timedelta(days=1),
                                periods=n_future_steps, freq='M') # Monthly st

    # Simulate micro data (assuming they remain relatively stable or follow a t
    # Updated to include new micro features based on their means/modes
    avg_fico = original_df['fico'].mean()
    avg_dti = original_df['dti'].mean()
    avg_annual_inc = original_df['annual_inc'].mean()
    avg_emp_length = original_df['emp_length'].mean() if 'emp_length' in origir
    avg_delinq_2y = original_df['delinq_2y'].mean() if 'delinq_2y' in original_
    avg_interest_rate = original_df['interest_rate'].mean() if 'interest_rate'

    # For categorical features, get the actual categorical dtype and its categc
    # This ensures consistent category sets between training and scenario data.
    categorical_feature_info = {}
    for col_name in ['emp_title', 'homeownership', 'loan_purpose', 'grade', 'su
        if col_name in original_df.columns:
            # Ensure the column is categorical in original_df first to get its
            if not pd.api.types.is_categorical_dtype(original_df[col_name]):
                original_df[col_name] = original_df[col_name].astype('category')

            categorical_dtype = original_df[col_name].dtype
            sampling_pool = original_df[col_name].cat.categories.tolist()
            # Fallback if no categories are found (e.g., all NaN in original_df
            if len(sampling_pool) == 0:
                if col_name == 'emp_title': sampling_pool = ['Other']
                elif col_name == 'homeownership': sampling_pool = ['RENT']
                elif col_name == 'loan_purpose': sampling_pool = ['debt_consoli
                elif col_name == 'grade': sampling_pool = ['B']
                elif col_name == 'sub_grade': sampling_pool = ['B3']
                elif col_name == 'term': sampling_pool = [36]

    else:
```

```

# Fallback if column is entirely missing from original_df
categorical_dtype = pd.CategoricalDtype(categories=['Unknown'], ordered=True)
if col_name == 'emp_title': sampling_pool = ['Other']
elif col_name == 'homeownership': sampling_pool = ['RENT']
elif col_name == 'loan_purpose': sampling_pool = ['debt_consolidation']
elif col_name == 'grade': sampling_pool = ['B']
elif col_name == 'sub_grade': sampling_pool = ['B3']
elif col_name == 'term': sampling_pool = [36]

categorical_feature_info[col_name] = {'dtype': categorical_dtype, 'sampling_pool': sampling_pool}

# New features
avg_earliest_credit_line = original_df['earliest_credit_line'].mean() if 'earliest_credit_line' in original_df else 0
avg_months_since_last_delinq = original_df['months_since_last_delinq'].mean() if 'months_since_last_delinq' in original_df else 0
avg_inquiries_last_12m = original_df['inquiries_last_12m'].mean() if 'inquiries_last_12m' in original_df else 0
avg_public_record_bankrupt = original_df['public_record_bankrupt'].mean() if 'public_record_bankrupt' in original_df else 0
avg_num_collections_last_12m = original_df['num_collections_last_12m'].mean() if 'num_collections_last_12m' in original_df else 0
avg_account_never_delinq_percent = original_df['account_never_delinq_percent'].mean() if 'account_never_delinq_percent' in original_df else 0
avg_loan_amount = original_df['loan_amount'].mean() if 'loan_amount' in original_df else 0

# Initialize stressed macro data
stressed_macro = pd.DataFrame({'date': future_dates})

if scenario_type == 'crisis':
    # Simulate a severe financial crisis / recession
    # Fed funds rates rise sharply then drop, unemployment spikes, CPI volatility
    stressed_macro['fed_funds'] = np.linspace(original_df['fed_funds'].max(), original_df['fed_funds'].min(), n_future_steps)
    stressed_macro['unemp_rate'] = np.linspace(original_df['unemp_rate'].min(), original_df['unemp_rate'].max(), n_future_steps)
    stressed_macro['cpi'] = np.linspace(original_df['cpi'].max(), original_df['cpi'].min(), n_future_steps)
    stressed_macro['ind_prod'] = np.linspace(original_df['ind_prod'].max(), original_df['ind_prod'].min(), n_future_steps)
    stressed_macro['hy_spread'] = np.linspace(original_df['hy_spread'].min(), original_df['hy_spread'].max(), n_future_steps)
    stressed_macro['real_gdp'] = np.linspace(original_df['real_gdp'].max(), original_df['real_gdp'].min(), n_future_steps)
elif scenario_type == 'covid_like':
    # Simulate a sudden shock like COVID-19
    # Sudden spike in unemployment, drop in GDP, then some recovery
    stressed_macro['fed_funds'] = np.array([original_df['fed_funds'].max()] + [original_df['fed_funds'].min()] * (n_future_steps - 1))
    unemp_spike = np.zeros(n_future_steps)
    unemp_spike[0:3] = [original_df['unemp_rate'].min() + 3, original_df['unemp_rate'].min() + 3, original_df['unemp_rate'].min() + 3]
    unemp_spike[3:] = np.linspace(unemp_spike[2], original_df['unemp_rate'].max(), n_future_steps - 3)
    stressed_macro['unemp_rate'] = unemp_spike
    stressed_macro['cpi'] = np.linspace(original_df['cpi'].mean(), original_df['cpi'].min(), n_future_steps)
    stressed_macro['ind_prod'] = np.array([original_df['ind_prod'].mean()] + [original_df['ind_prod'].min()] * (n_future_steps - 1))
    stressed_macro['hy_spread'] = np.array([original_df['hy_spread'].mean()] + [original_df['hy_spread'].min()] * (n_future_steps - 1))
    stressed_macro['real_gdp'] = np.array([original_df['real_gdp'].mean()] + [original_df['real_gdp'].min()] * (n_future_steps - 1))
else:
    # Default to a mild recession scenario
    stressed_macro['fed_funds'] = np.linspace(original_df['fed_funds'].mean(), original_df['fed_funds'].min(), n_future_steps)
    stressed_macro['unemp_rate'] = np.linspace(original_df['unemp_rate'].mean(), original_df['unemp_rate'].min(), n_future_steps)
    stressed_macro['cpi'] = np.linspace(original_df['cpi'].mean(), original_df['cpi'].min(), n_future_steps)
    stressed_macro['ind_prod'] = np.linspace(original_df['ind_prod'].mean(), original_df['ind_prod'].min(), n_future_steps)

```

```

stressed_macro['hy_spread'] = np.linspace(original_df['hy_spread'].mean(),
stressed_macro['real_gdp'] = np.linspace(original_df['real_gdp'].mean(),

# Create a dummy loan_df for the future period (assuming a constant number
n_future_loans = n_samples // (3000 // n_future_steps) # Adjust based on or
future_loan_df = pd.DataFrame({
    'issue_d': np.random.choice(future_dates, n_future_loans),
    'fico': np.random.normal(avg_fico, 20, n_future_loans),
    'dti': np.random.normal(avg_dti, 5, n_future_loans),
    'annual_inc': np.random.lognormal(np.log(avg_annual_inc), 0.2, n_future_loans),
    'emp_length': np.random.randint(max(0, int(avg_emp_length - 2)), int(avg_emp_length + 2), n_future_loans),
    'delinq_2y': np.random.randint(max(0, int(avg_delinq_2y - 1)), int(avg_delinq_2y + 1), n_future_loans),
    'interest_rate': np.random.normal(avg_interest_rate, 2, n_future_loans),
    'earliest_credit_line': np.random.randint(max(1990, int(avg_earliest_credit_line - 10)), int(avg_earliest_credit_line + 10), n_future_loans),
    'months_since_last_delinq': np.random.randint(0, int(original_df['months_since_last_delinq'].max() + 1), n_future_loans),
    'inquiries_last_12m': np.random.randint(max(0, int(avg_inquiries_last_12m - 1)), int(avg_inquiries_last_12m + 1), n_future_loans),
    'public_record_bankrupt': np.random.randint(0, int(avg_public_record_bankrupt + 1), n_future_loans),
    'num_collections_last_12m': np.random.randint(0, int(avg_num_collections_last_12m + 1), n_future_loans),
    'account_never_delinq_percent': np.random.uniform(max(0.0, avg_account_never_delinq_percent - 0.1), min(100.0, avg_account_never_delinq_percent + 0.1), n_future_loans),
    'loan_amount': np.random.normal(avg_loan_amount, avg_loan_amount * 0.1, n_future_loans)
})

# Add categorical columns to future_loan_df, sampling from known categories
for col_name in ['emp_title', 'homeownership', 'loan_purpose', 'grade', 'sub_grade', 'term']:
    if col_name in categorical_feature_info:
        future_loan_df[col_name] = np.random.choice(categorical_feature_info[col_name], n_future_loans)
    else:
        # If a categorical column was not found in original_df, add it with a default value
        if col_name == 'emp_title': default_cat = 'Other'
        elif col_name == 'homeownership': default_cat = 'RENT'
        elif col_name == 'loan_purpose': default_cat = 'debt_consolidation'
        elif col_name == 'grade': default_cat = 'B'
        elif col_name == 'sub_grade': default_cat = 'B3'
        elif col_name == 'term': default_cat = 36
        future_loan_df[col_name] = pd.Series([default_cat] * n_future_loans)

# Temporal Alignment: "Point-in-Time" Join
stressed_df = pd.merge_asof(future_loan_df.sort_values('issue_d'),
                             stressed_macro.sort_values('date'),
                             left_on='issue_d', right_on='date',
                             direction='backward').drop('date', axis=1)

return stressed_df

```

```

def run_stress_test(model_results, scenario_df, model_name="Macro-Augmented"):
    """
    Runs the stress test on a given scenario data using a specified trained model.
    """
    if model_name not in model_results:

```

```

        raise ValueError(f"Model '{model_name}' not found in model_results.")

# Updated micro_feats list for the new feature set
micro_feats = ['fico', 'dti', 'annual_inc', 'emp_length', 'delinq_2y', 'int
               'emp_title', 'earliest_credit_line', 'months_since_last_delinq',
               'inquiries_last_12m', 'homeownership', 'public_record_bankrupt',
               'num_collections_last_12m', 'account_never_delinq_percent',
               'term', 'loan_purpose', 'grade', 'sub_grade']
macro_feats = ['fed_funds', 'unemp_rate', 'cpi', 'ind_prod', 'hy_spread', '

# Retrain the Macro-Augmented model on the full 'real_data' (not the initial)
# This is crucial for categorical consistency between training and scenario
X_full_real = data[micro_feats + macro_feats]
y_full_real = data['default']

# Ensure categorical columns in X_full_real are treated as such before training
categorical_cols = ['emp_title', 'homeownership', 'loan_purpose', 'grade',
                    'sub_grade']
for col in categorical_cols:
    if col in X_full_real.columns:
        X_full_real[col] = X_full_real[col].astype('category')

stressed_clf = XGBClassifier(enable_categorical=True, n_estimators=100, max_depth=
                             6, learning_rate=0.1)
stressed_clf.fit(X_full_real, y_full_real)

# Prepare the scenario data for prediction
# Ensure scenario_df columns have the exact same categorical dtypes as X_full_real
for col in categorical_cols:
    if col in scenario_df.columns:
        if col in X_full_real.columns and pd.api.types.is_categorical_dtype(
            X_full_real[col]):
            scenario_df[col] = pd.Categorical(scenario_df[col], categories=
                X_full_real[col].cat.categories)
        else:
            scenario_df[col] = scenario_df[col].astype('category')

X_scenario = scenario_df[micro_feats + macro_feats]

# Predict probabilities under stress
stress_pred_proba = stressed_clf.predict_proba(X_scenario)[ :, 1]

scenario_df['predicted_default_proba'] = stress_pred_proba

return scenario_df

```

```

n_samples = 3000 # Use the same n_samples as in build_integrated_pipeline for c
# Create a 'Crisis' scenario
crisis_scenario_df = create_stress_scenario_data(data, n_future_steps=24, scenar
# Run stress test for the crisis scenario
crisis_results = run_stress_test(model_performance, crisis_scenario_df, model_r

```

```

# Calculate average default probability under crisis
average_crisis_default_proba = crisis_results['predicted_default_proba'].mean()
print(f"Average predicted default probability under 'Crisis' scenario: {average_crisis_default_proba}")

# Create a 'COVID-like' scenario
covid_scenario_df = create_stress_scenario_data(data, n_future_steps=24, scenario='COVID-like')

# Run stress test for the COVID-like scenario
covid_results = run_stress_test(model_performance, covid_scenario_df, model_name='Crisis')

# Calculate average default probability under COVID-like scenario
average_covid_default_proba = covid_results['predicted_default_proba'].mean()
print(f"Average predicted default probability under 'COVID-like' scenario: {average_covid_default_proba}")

# Create a 'Mild Recession' scenario
mild_recession_scenario_df = create_stress_scenario_data(data, n_future_steps=24, scenario='Mild Recession')

# Run stress test for the Mild Recession scenario
mild_recession_results = run_stress_test(model_performance, mild_recession_scenario_df, model_name='Mild Recession')

# Calculate average default probability under Mild Recession scenario
average_mild_recession_default_proba = mild_recession_results['predicted_default_proba'].mean()
print(f"Average predicted default probability under 'Mild Recession' scenario: {average_mild_recession_default_proba}")

```

```

Average predicted default probability under 'Crisis' scenario: 0.2687
Average predicted default probability under 'COVID-like' scenario: 0.4317
Average predicted default probability under 'Mild Recession' scenario: 0.5224

```

```

n_samples = 3000 # Use the same n_samples as in build_integrated_pipeline for crisis scenario

# Create a 'Crisis' scenario
crisis_scenario_df = create_stress_scenario_data(data, n_future_steps=24, scenario='Crisis')

# Run stress test for the crisis scenario
crisis_results = run_stress_test(model_performance, crisis_scenario_df, model_name='Crisis')

# Calculate average default probability under crisis
average_crisis_default_proba = crisis_results['predicted_default_proba'].mean()
print(f"Average predicted default probability under 'Crisis' scenario: {average_crisis_default_proba}")

# Create a 'COVID-like' scenario
covid_scenario_df = create_stress_scenario_data(data, n_future_steps=24, scenario='COVID-like')

# Run stress test for the COVID-like scenario
covid_results = run_stress_test(model_performance, covid_scenario_df, model_name='COVID-like')

# Calculate average default probability under COVID-like scenario
average_covid_default_proba = covid_results['predicted_default_proba'].mean()
print(f"Average predicted default probability under 'COVID-like' scenario: {average_covid_default_proba}")

# Create a 'Mild Recession' scenario
mild_recession_scenario_df = create_stress_scenario_data(data, n_future_steps=24, scenario='Mild Recession')

```

```

mild_recession_scenario_df = create_stress_scenario_data(data, n_future_steps=

# Run stress test for the Mild Recession scenario
mild_recession_results = run_stress_test(model_performance, mild_recession_sce

# Calculate average default probability under Mild Recession scenario
average_mild_recession_default_proba = mild_recession_results['predicted_defaul
print(f"Average predicted default probability under 'Mild Recession' scenario:

Average predicted default probability under 'Crisis' scenario: 0.2591
Average predicted default probability under 'COVID-like' scenario: 0.4587
Average predicted default probability under 'Mild Recession' scenario: 0.4534

```

✓ Visualizing Stress Test Results

Let's visualize the predicted default probabilities over time for each stress scenario.

```

plt.figure(figsize=(15, 7))

sns.lineplot(x='issue_d', y='predicted_default_proba', data=crisis_results, label='Crisis Scenario')
sns.lineplot(x='issue_d', y='predicted_default_proba', data=covid_results, label='COVID-like Scenario')
sns.lineplot(x='issue_d', y='predicted_default_proba', data=mild_recession_results, label='Mild Recession Scenario')

plt.title('Predicted Default Probability Over Time Under Various Stress Scenarios')
plt.xlabel('Date')
plt.ylabel('Predicted Default Probability')
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend()
plt.tight_layout()
plt.show()

```

